

EXPERIMENT – 1

Name: Arjun Gore

Class: D20A - 44

Aim

Cryptography in Blockchain – Merkle Root Tree Hash

Theory

1. Cryptographic Hash Functions in Blockchain

A cryptographic hash function converts input data of any size into a fixed-length string called a **hash**. In blockchain, **SHA-256** is the most widely used hashing algorithm because it is deterministic, collision-resistant, and one-way in nature. Even a small change in the input produces a completely different output (avalanche effect), making hashing essential for maintaining data integrity and blockchain security.

2. What is a Merkle Tree?

A **Merkle Tree** (Hash Tree) is a hierarchical data structure used to efficiently verify the integrity of a large number of transactions. Each transaction is hashed individually, and the hashes are combined in pairs until a single hash remains at the top. This final hash is called the **Merkle Root**, which uniquely represents all the transactions contained in the block.

3. Cryptographic Puzzle and the Golden Nonce

A cryptographic puzzle is the basis of the **Proof-of-Work (PoW)** consensus mechanism. Miners repeatedly combine block data with different **nonce** values and calculate the hash until they find

one that satisfies the required difficulty (usually a specified number of leading zeros). The successful nonce is called the **Golden Nonce**, and the miner who discovers it earns the right to add the next block to the blockchain.

4. How Does a Merkle Tree Work?

The Merkle Tree is built by first hashing every transaction individually. These hashes are grouped in pairs and hashed again to form parent nodes. If the number of hashes is odd, the last hash is duplicated before pairing. This process continues until only one hash remains—the **Merkle Root**—which is stored in the block header and acts as a fingerprint for all transactions in the block.

5. Benefits of Merkle Tree

- Fast and efficient verification of transactions.
- Supports lightweight verification for mobile and lightweight blockchain clients.
- Detects any data tampering immediately.
- Reduces storage and bandwidth requirements by storing only the Merkle Root instead of all transaction data.

6. Use Cases of Merkle Tree

Merkle Trees are widely used in:

- Blockchain platforms such as Bitcoin and Ethereum.
- Peer-to-peer file-sharing systems like BitTorrent.
- Git version control system.
- Distributed databases and cloud storage systems for data verification.

Programs

Program 1: SHA-256 Hash Generation

```
import hashlib

def create_hash(string):
    hash_object = hashlib.sha256()
    hash_object.update(string.encode('utf-8'))
    hash_string = hash_object.hexdigest()
    return hash_string

input_string = input("Enter a string: ")
hash_result = create_hash(input_string)
print("Hash:", hash_result)
```

Program 2: Hash Generation using Input String and Nonce

```
import hashlib

input_string = input("Enter a string: ")
nonce = input("Enter the nonce: ")

hash_string = input_string + nonce

hash_object = hashlib.sha256(hash_string.encode('utf-8'))
hash_code = hash_object.hexdigest()

print("Hash Code:", hash_code)
```

Program 3: Finding the Golden Nonce

```
import hashlib

def find_nonce(input_string, num_zeros):
    nonce = 0
    hash_prefix = '0' * num_zeros

    while True:
        hash_string = input_string + str(nonce)
        hash_object = hashlib.sha256(hash_string.encode('utf-8'))
        hash_code = hash_object.hexdigest()

        if hash_code.startswith(hash_prefix):
            print("Hash:", hash_code)
```

```

        return nonce

    nonce += 1

input_string = "Arjun"
num_zeros = 3

expected_nonce = find_nonce(input_string, num_zeros)

print("Input String:", input_string)
print("Leading Zeros:", num_zeros)
print("Expected Nonce:", expected_nonce)

```

Program 4: Merkle Tree Construction

```

import hashlib

def build_merkle_tree(transactions):

    if len(transactions) == 0:
        return None

    hashed_transactions = []

    print("==== TRANSACTION HASHES =====")

    for transaction in transactions:
        hash_transaction = hashlib.sha256(
            transaction.encode('utf-8')
        ).hexdigest()

        hashed_transactions.append(hash_transaction)

        print("Transaction:", transaction)
        print("Hash:", hash_transaction)
        print()

    transactions = hashed_transactions

    tree = [transactions.copy()]
    level = 1

    while len(transactions) > 1:

```

```

if len(transactions) % 2 != 0:
    transactions.append(transactions[-1])

new_transactions = []

print("==== MERKLE TREE LEVEL", level, "====")

for i in range(0, len(transactions), 2):
    combined = transactions[i] + transactions[i + 1]

    hash_combined = hashlib.sha256(
        combined.encode('utf-8')
    ).hexdigest()

    new_transactions.append(hash_combined)

    print("Hash 1:", transactions[i])
    print("Hash 2:", transactions[i + 1])
    print("Combined Hash:", hash_combined)
    print()

    transactions = new_transactions
    tree.append(transactions.copy())
    level += 1

return transactions[0], tree

```

```

transactions = [
    "Alice -> Bob : $200",
    "Bob -> Dave : $500",
    "Dave -> Eve : $100",
    "Eve -> Alice : $300",
    "Roo -> Bob : $50"
]

merkle_root, merkle_tree = build_merkle_tree(transactions)

print("===== MERKLE TREE =====")

for i, level in enumerate(merkle_tree):
    print("Level", i, ":")

```

```
for j, hash_value in enumerate(level):
    print("H{} = {}".format(j + 1, hash_value))

print()

print("=====")
print("Merkle Root:", merkle_root)
```

Output

Program 1

```
Enter a string: Arjun
Hash: 2bb0c46073ba8fe53d965ee993c2463398cb9df296f72524217d65696f9b265d
```

Program 2

```
Enter a string: Arjun
Enter the nonce: 15
Hash Code:
eea243c07ccc3c1e5c03d4f7c6a7202c1a9702e193aa0c4feae0c7d8d4ad2c32
```

Program 3

```
Hash: 000042543f55b76f2707d82846d7ab0e00056d3f11a67a18239edf483bd2ca6a
Input String: Arjun
Leading Zeros: 3
Expected Nonce: 159
```

Program 4

Transaction hashes generated successfully.

Merkle Tree Levels:

Level 0 → Transaction Hashes

Level 1 → Parent Hashes

Level 2 → Parent Hashes

Level 3 → Merkle Root

Merkle Root:

96ee90df2272d24c0bcf4c67e152c257dc45bce4560221125b0750cead7b27b0

Conclusion

This experiment demonstrates the core cryptographic concepts used in blockchain technology.

- **SHA-256 hashing** generates a secure and unique fingerprint for any input, ensuring data integrity.
- **Nonce-based hashing** simulates the Proof-of-Work mechanism by generating different hashes until the required condition is satisfied.
- **Proof-of-Work** illustrates how miners find the Golden Nonce through repeated hashing, making blockchain secure against tampering.
- **Merkle Tree construction** efficiently summarizes and verifies a large number of transactions using a single Merkle Root Hash.

Overall, these implementations highlight how cryptographic hashing, Proof-of-Work, and Merkle Trees work together to provide security, immutability, and efficient verification in blockchain systems.